

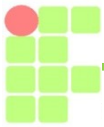
Instituto Federal de Educação, Ciência e Tecnologia do Piauí
IFPI Campus Campo Maior
Coordenação de Informática
Técnico em Informática Conc./Subs. (Módulo III)



Algoritmos e Ling. de Programação

Tecnólogo ADS

M.Sc Nairon Viana
nairon.viana@ifpi.edu.br



Disciplina: Estruturas de Dados



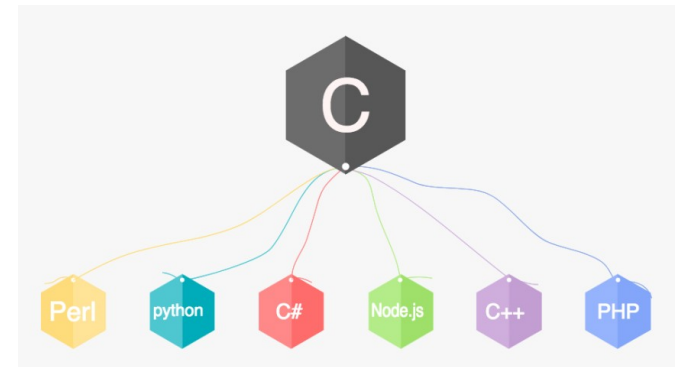
Conteúdo – Modularização em C

- **Parte I:** Programação Modular
- **Parte II:** Módulos em C
 - Procedimentos (sem retorno)
 - Funções (com retorno)
- **Parte III:** Bibliotecas Externas do Sistema/Usuário
 - Bibliotecas de Módulos do Sistema
 - Bibliotecas de Módulos do Usuário
 - Exemplos práticos de Módulos em Linguagem C

Disciplina: Estruturas de Dados

Conteúdo – Modularização em C

Parte I: Programação Modular



Parte I – Linguagem C – Programação Modular

Conceito

– O que é modularizar um programa?

Dividir o seu código em partes, cada uma realizando uma tarefa específica, podendo ser chamada por um **módulo principal**.

Em programas mais antigos, todo o código-fonte era concentrado em um único bloco/local, o que provocava uma série de problemas especialmente em códigos extensos

Sem Modularização

Bloco Principal (engloba todos os códigos)

Bloco 1

Bloco 2

Bloco 3

Bloco 4

Com Modularização

Bloco Principal

Separação de código

b1() ← Bloco 1 – nome: **b1()**

b2() ← Bloco 2 – nome: **b2()**

b3() ← Bloco 3 – nome: **b3()**

b4() ← Bloco 4 – nome: **b4()**

Parte I – Linguagem C – Programação Modular

Conceito

– O que é modularizar um programa?

Na **modularização** o bloco/módulo principal (em C, o bloco **main**) substitui a chamada de um **trecho de código inteiro** pelo **nome** do módulo onde esse código está implementado

O **módulo** pode ser chamado quantas vezes forem necessárias, e sempre que referenciado pelo seu nome, o trecho de código é executado no ponto em que foi chamado

Em Linguagem C:

```
int main(){  
  modulo1();  
  modulo2();  
  (...)  
  modulo3();  
  modulo2();  
}
```

```
void modulo1(){  
  // Comandos do  
  // Módulo 1  
}
```

```
void modulo2(){  
  // Comandos do  
  // Módulo 2  
}
```

```
void modulo3(){  
  // Comandos do  
  // Módulo 3  
}
```

O módulo **main()** é o único que é **obrigatório** para a execução do programa em C

Parte I – Linguagem C – Programação Modular

Conceito

– O que é modularizar um programa?

Separar código do módulo **principal (main)** e demais módulos do programa

```
/** Módulo Principal */
```

```
int main(){
```

```
    printf("Bem-Vindo! ");
    printf("Olá!");
```

```
    int x = 10;
    int y = 5;

    if(x>y){
        printf("x é maior");
    }else{
        if(y>x){
            printf("x é maior");
        }else{
            printf("x e y são iguais");
        }
    }
}
```

```
    return 0;
```

```
}
```

Não-Modularizado

```
void bemVindo(){
    printf("Bem-Vindo! ");
    printf("Olá!");
}
```

```
void maiorMenor(){
```

```
    int x = 10;
    int y = 5;
```

```
    if(x>y){
        printf("x é maior");
    }else{
        if(y>x){
            printf("x é maior");
        }else{
            printf("x e y são iguais");
        }
    }
}
```

```
}
```

Modularizado

```
/** Módulo Principal */
int main(){
    bemVindo();
    maiorMenor();

    return 0;
}
```

Parte I – Linguagem C – Programação Modular

Conceito

– O que é modularizar um programa?

Módulos são também chamados **sub-rotinas** ou **sob-programas**, já que em um módulo é possível:

- Declarar variáveis (homogêneas, heterogêneas, simples, compostas)
- Utilizar comandos de entrada/saída
- Utilizar outras estruturas algorítmicas (decisão, repetição, etc...)
- Realizar operações aritméticas/lógicas
- Chamar outros módulos

Parte I – Linguagem C – Programação Modular

Conceito

– Exemplo

```
/** Módulo Principal */
int main(){

    calculaMedia();

    return 0;
}
```

```
void calculaMedia(){
    float n1, n2, m;
    // Chama Módulo
    mensagem();
    printf("Digite n1: ");
    scanf("%f", &n1);
    printf("Digite n2: ");
    scanf("%f", &n2);
    m = (n1+n2)/2;
    printf("Média: %.2f", m);
    if(m>7.0){
        printf("APROVADO!");
    }
}
```

```
void mensagem(){
    printf("Bem-Vindo ");
    printf("à Programação ");
    printf("Modular \n");
}
```

EXECUÇÃO

– Ordem de execução:

1. Módulo **main()** chama módulo **calculaMedia()**
2. Módulo **calculaMedia()** declara variáveis e chama Módulo **mensagem()**
3. Módulo **mensagem** executa e volta para Módulo **calculaMedia()**
4. Módulo **calculaMedia()** executa e volta para **main()** que finaliza com **return 0**

```
Bem-Vindo à Programação Modular
Digite n1: 10
Digite n2: 8
Média: 9.00APROVADO!

...Program finished with exit code 0
Press ENTER to exit console.
```


Parte I – Linguagem C – Programação Modular

Exemplo em C – Sem Modularização

```
#include<stdio.h>
#include<stdlib.h>

/** Declaração de Variáveis */
float n1, n2;
float s, su, p, d;

/** Módulo Principal */
int main(){

    printf("Digite n1: ");
    scanf("%f", &n1);
    printf("Digite n2: ");
    scanf("%f", &n2);

    printf("Valor de n1: %.2f\n", n1);
    printf("Valor de n2: %.2f\n", n2);

    s = (n1+n2);
    printf("Soma: %.2f\n", s);

    su = (n1-n2);
    printf("Subtração: %.2f\n", su);

    p = (n1*n2);
    printf("Produto: %.2f\n", p);

    d = (n1/n2);
    printf("Divisão: %.2f\n", d);

    /** Ler novamente os valores */
    printf("Digite n1: ");
    scanf("%f", &n1);
    printf("Digite n2: ");
    scanf("%f", &n2);

    /** Mostrar novamente os valores */
    printf("Valor de n1: %.2f\n", n1);
    printf("Valor de n2: %.2f\n", n2);

    /** Calcular e mostrar a soma */
    s = (n1+n2);
    printf("Soma: %.2f\n", s);

    return 0;
}
```

Parte I – Linguagem C – Programação Modular

```
/** Variáveis */
float n1, n2;
float s, su, p, d;
```

Com Modularização

```
/** Módulo Principal */
int main(){
```

```
    leitura();
```

```
    mostrar();
```

```
    soma();
```

```
    subtracao();
```

Módulo Principal

```
    produto();
```

```
    divisao();
```

```
    /** Ler novamente os valores */
```

```
    leitura();
```

```
    /** Mostrar novamente os valores */
```

```
    mostrar();
```

```
    /** Calcular e mostrar a soma */
```

```
    soma();
```

```
    return 0;
```

```
}
```

```
void leitura(){
    printf("Digite n1: ");
    scanf("%f", &n1);
    printf("Digite n2: ");
    scanf("%f", &n2);
}
```

```
void mostrar(){
    printf("Valor de n1: %.2f\n", n1);
    printf("Valor de n2: %.2f\n", n2);
}
```

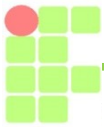
```
void soma(){
    s = (n1+n2);
    printf("Soma: %.2f\n", s);
}
```

```
void subtracao(){
    su = (n1-n2);
    printf("Subtração: %.2f\n", su);
}
```

```
void produto(){
    p = (n1*n2);
    printf("Produto: %.2f\n", p);
}
```

```
void divisao(){
    d = (n1/n2);
    printf("Divisão: %.2f\n", d);
}
```

Módulos Secundários



Parte I – Linguagem C – Programação Modular

Conceito

– Vantagens da Modularização

Escrever código modularizado traz uma série de vantagens:

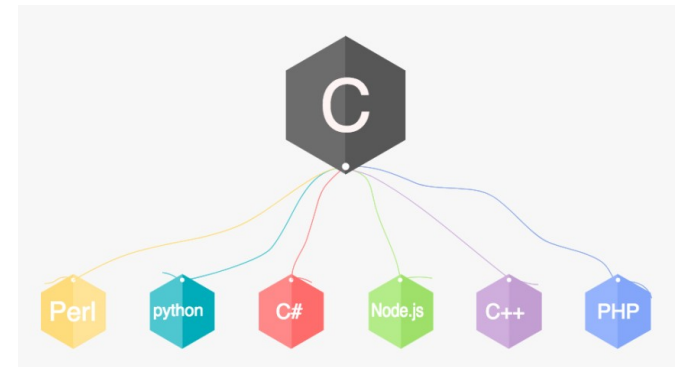
- Separação do Código: cada módulo pode ser desenvolvido de maneira independente, separado do programa principal
- Facilidade de Manutenção: modificar apenas uma parte do código é mais fácil
- Legibilidade: com o código separado sua apresentação fica mais fácil de ser lida
- Menos linhas de Código: o tamanho do programa principal pode ser reduzido
- Reutilização de Módulos: um módulo pode ser escrito para vários outros tipos de programas e re-aproveitado (ex.: módulos do ambiente C: **printf**, **scanf**)



Disciplina: Estruturas de Dados

Conteúdo – Modularização em C

Parte II: Módulos em C

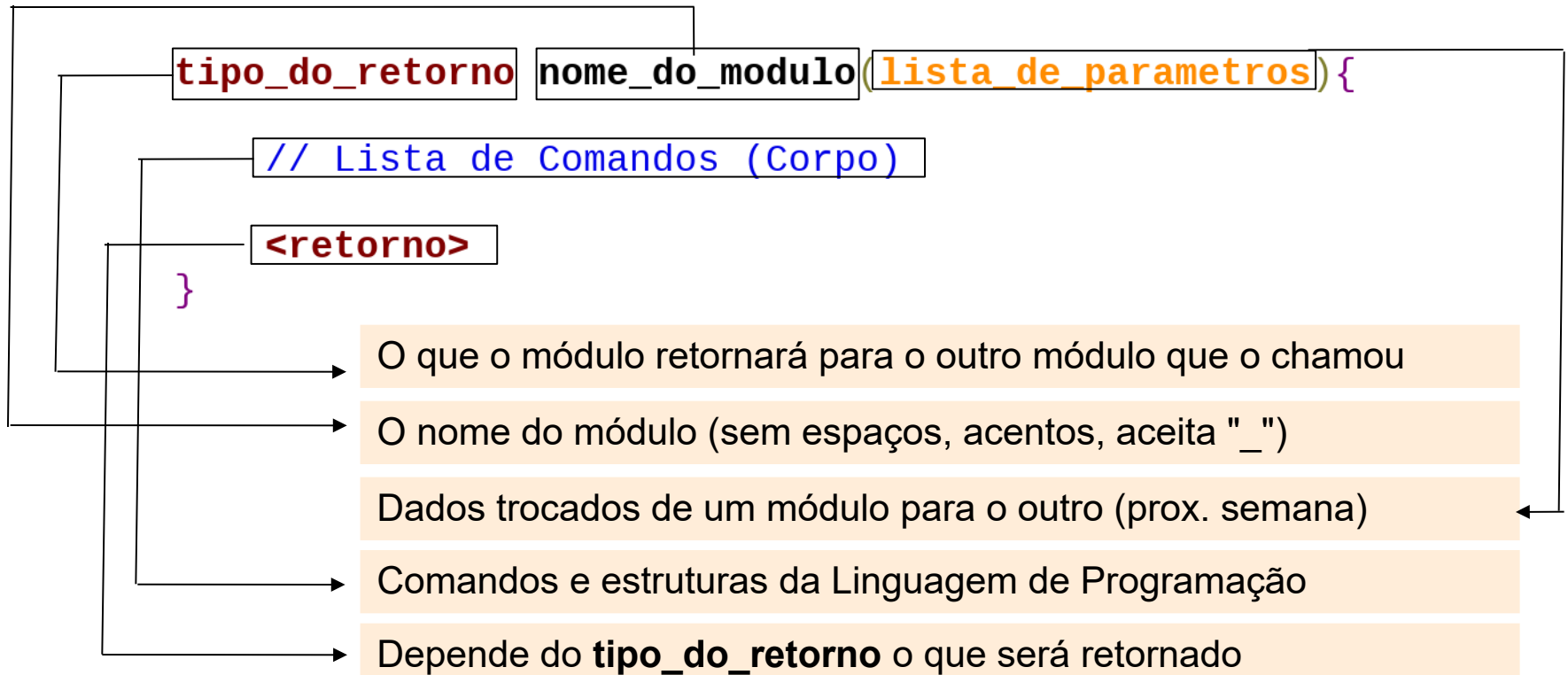


Parte II – Tipos de Módulos em Linguagem C

Definições

– Estrutura Geral de um Módulo

Módulos ou **sub-rotinas/sob-programas** apresentam de uma maneira geral o seguinte formato/estrutura/sintaxe:



Parte II – Tipos de Módulos em Linguagem C

Definições

– Estrutura Geral de um Módulo (Exemplos)

```
void soma(){
    int x, y, s;
    scanf("%d", &x);
    scanf("%d", &y);
    s = (x+y);
    printf("%d", s);
}
```

Sem retorno e
sem parâmetros

```
void soma(int x, int y){
    int s = (x+y);
    printf("Soma: %.d", s);
}
```

Sem retorno e
com parâmetros

```
int soma(int x, int y){
    int s = (x+y);
    return s;
}
```

Com retorno e
com parâmetros

```
float quadrado(float x){
    float q = x*x;
    return q;
}
```

Com retorno e
com parâmetros

```
void ola(){
    printf("Olá!\n");
}
```

Sem retorno e
sem parâmetros

```
float valorPi(){
    return 3.14;
}
```

Com retorno e
sem parâmetros

Parte II – Tipos de Módulos em Linguagem C

Definições

– Procedimentos e Funções

O **tipo do retorno** e o **retorno** definem os dois tipos de módulos existentes: funções e procedimentos

Diferem-se na **forma como são chamados** pelo módulo principal

Quando o módulo é chamado e nenhum retorno é dado ao módulo que o chamou, dizemos que o módulo é um **procedimento**. Exemplo:

```
int main(){  
    soma();  
    return 0;  
}
```

```
void soma(){  
    int x, y, s;  
    scanf("%d", &x);  
    scanf("%d", &y);  
    s = (x+y);  
    printf("%d", s);  
}
```

O módulo (procedimento) **soma()** não retorna nada para o módulo **main()**, apenas realiza uma tarefa delegada por ele

Parte II – Tipos de Módulos em Linguagem C

Definições

– Procedimentos e Funções

O **tipo do retorno** e o **retorno** definem os dois tipos de módulos existentes: funções e procedimentos

Quando o módulo é chamado e um retorno é passado deste para dentro do módulo que o chamou (sendo armazenado em uma variável) dizemos que o módulo é uma **função**. Exemplo:

```
int main(){  
    int    x1 = soma();  
    float  x2 = valorPi();  
  
    printf("Soma: %d\n", x1);  
    printf("Pi: %.2f", x2);  
  
    return 0;  
}
```

```
int soma(){  
    int x=10, y=4;  
    int s = (x+y);  
    return s;  
}  
  
float valorPi(){  
    return 3.14;  
}
```

Os módulo (função) **soma()** e **valorPi()** **retornam** para o módulo **main()**, os valores já processados. Main armazena os valores em suas próprias variáveis

Parte II – Tipos de Módulos em Linguagem C

Definições

– Procedimentos e Funções

Em Linguagem C, um **procedimento** geralmente é um módulo que realiza uma ação independente, sem retornar nenhum valor; o nome do módulo é precedido pela palavra **void**

Módulo principal (**main**)

```
int main(){  
    calculaMedia();  
    return 0;  
}
```

Procedimento (sem retorno - **void**)

```
void calculaMedia(){  
    float x=5, y=8;  
    float m=(x+y)/2;  
    printf("Média: %.2f", m);  
}
```

Parte II – Tipos de Módulos em Linguagem C

Definições

– Procedimentos e Funções

Já **funções** devem obrigatoriamente **retornar** um valor de algum tipo definido para o módulo que a chamou;

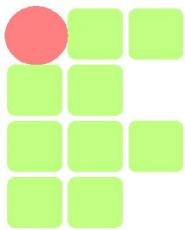
O nome do módulo é precedido pelo nome do tipo retornado e a última linha da função deve ter o valor retornado precedido da palavra reservada **return**

Módulo principal (**main**)

```
int main(){  
    float md = calculaMedia();  
    printf("Média: %.2f", md);  
  
    return 0;  
}
```

Função (com retorno – **return**)

```
float calculaMedia(){  
    float x=5, y=8;  
    float m=(x+y)/2;  
    return m;  
}
```



Instituto Federal de Educação, Ciência e Tecnologia do Piauí
IFPI Campus Campo Maior
Coordenação de Informática
Técnico em Informática Conc./Subs. (Módulo III)



Estruturas de Dados

Atividades Não-Presenciais

M.Sc Nairon Viana
nairon.viana@ifpi.edu.br